

Desarrollar Software a Partir de la Integración de sus Módulos y Componentes



Tecnólogo en Análisis y Desarrollo de Software Ficha – 3118562

Instructor: Henry Alfonso Garzón Sanchez

Contenido

Introducción	4
Casos de Uso	5
Diagrama de Clases.....	6
Entidades principales	6
Nacionalidad.	6
Persona.....	6
Rol.....	7
Usuario.....	7
DTOs (Objetos de Transferencia de Datos):.....	7
LoginDto.....	7
RegistroDto.....	7
UsuarioResponseDto.....	7
Diagrama de Paquetes.....	8
Diagrama de Componentes	9
Mecanismos de Seguridad	11
Capas del Sistema	11
Metodología de Desarrollo	12
Fases del Desarrollo.....	13

Mapa de Navegación.....	14
Pruebas Unitarias de los Módulos.....	15
Módulo 1 – Nacionalidades	15
Prueba 1 – Listar todas las nacionalidades.	15
Prueba 2 — Obtener nacionalidad por ID.....	16
Prueba 3 — Obtener nacionalidad con ID inexistente.....	17
Módulo 2: Personas.....	17
Prueba 4 — Listar todas las personas	17
Prueba 5 — Crear persona con datos completos	18
Prueba 6 — Crear persona sin campos requeridos	19
Prueba 7 — Actualizar persona existente	20
Prueba 8 — Eliminar persona	20
Módulo 3: Roles y Autenticación	21
Prueba 9 — Listar todos los roles	21
Prueba 10 — Login con credenciales válidas	21
Prueba 11 — Login sin credenciales	22
Prueba 12 — Login con credenciales inválidas	23

Introducción

Para esta evidencia se desarrolló el sistema **NotePlus Taller**, una aplicación web de gestión de personas que hace uso de algunas de las tablas utilizadas en el proyecto NotePlus. El sistema permite registrar usuarios, gestionar personas con sus datos personales y administrar el catálogo de nacionalidades, implementando autenticación y control de acceso.

El desarrollo se realizó aplicando una metodología basada en inteligencia artificial mediante prompts, utilizando **Claude Code** como herramienta principal para la generación del código fuente. El stack tecnológico utilizado fue **Next.js** con **TypeScript** para el backend y frontend, **Supabase** como base de datos en la nube y **Vercel** como plataforma de despliegue.

El documento presenta los artefactos de diseño del sistema (casos de uso, diagrama de clases, diagrama de paquetes, diagrama de componentes, mapa de navegación), la arquitectura por capas, los mecanismos de seguridad implementados, la metodología de desarrollo y el informe de pruebas unitarias realizadas con Postman sobre los endpoints desplegados en producción.

Casos de Uso

Para este taller se definieron dos actores principales:

Usuario No Autenticado: persona que accede al sistema sin credenciales. Puede registrarse creando una cuenta nueva o iniciar sesión con credenciales existentes.

Usuario Autenticado: persona que ha iniciado sesión correctamente. Tiene acceso completo al módulo de gestión de personas y nacionalidades, y puede cerrar sesión cuando lo desee.

Los casos de uso del sistema son:

Caso de Uso	Actor	Descripción
Registrarse	No Autenticado	Crea una Persona y un Usuario asociado con contraseña
Iniciar Sesión	No Autenticado	Valida credenciales y establece sesión
Cerrar Sesión	Autenticado	Elimina la sesión activa
Ver Personas	Autenticado	Lista todas las personas registradas
Crear Persona	Autenticado	Registra una nueva persona en el sistema
Editar Persona	Autenticado	Modifica los datos de una persona existente
Eliminar Persona	Autenticado	Realiza baja lógica de una persona
Ver Nacionalidades	Autenticado	Lista las nacionalidades disponibles

Caso de Uso	Actor	Descripción
Crear Nacionalidad	Autenticado	Agrega una nueva nacionalidad al catálogo

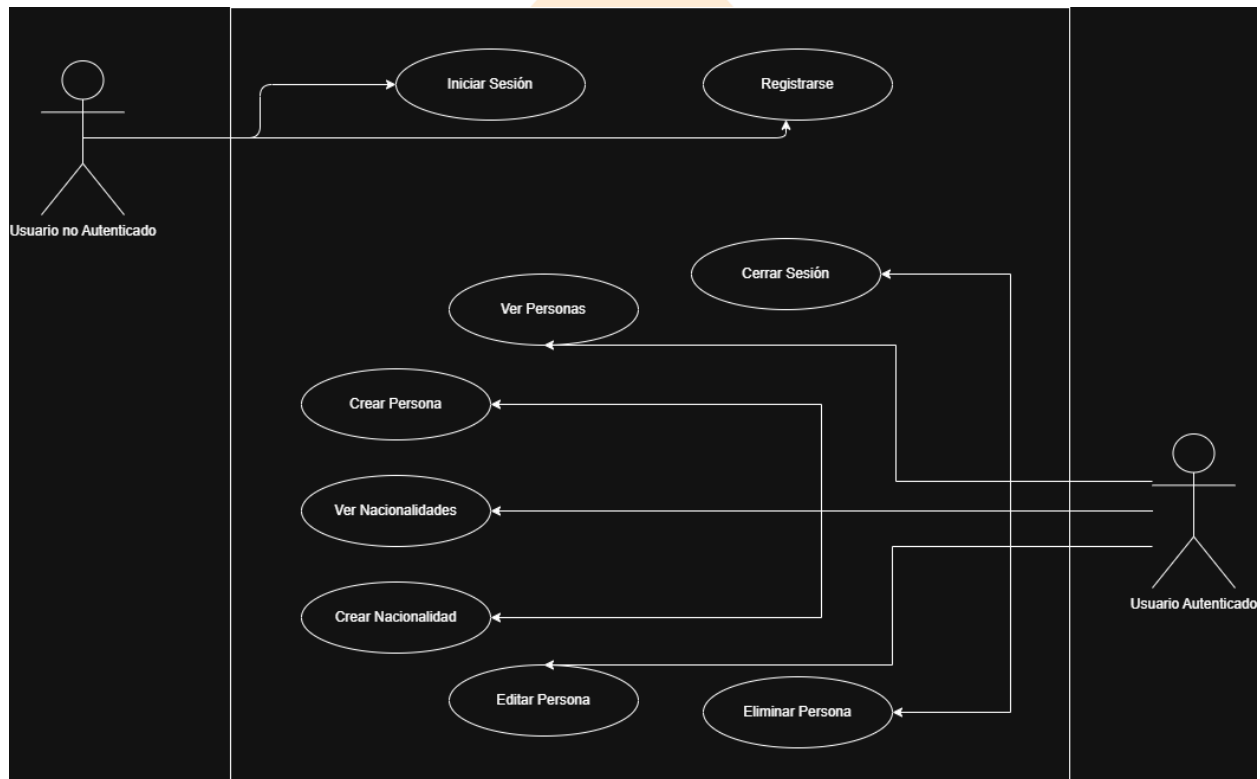


Diagrama de Clases

El diagrama de clases representa las entidades involucradas dentro del sistema y sus relaciones. Para este taller se utilizó la extensión de VSCode llamada **classdiagram-ts**.

Entidades principales

Nacionalidad.

Catálogo de nacionalidades con IDNacionalidad y NombreNacionalidad

Persona.

Entidad central con datos personales, documento de identidad, contacto y referencia a Nacionalidad.

Rol.

Catálogo de roles del sistema con IDRol, NombreRol y Descripción

Usuario.

Entidad de autenticación que referencia a Persona y Rol, almacena credenciales.

DTOs (Objetos de Transferencia de Datos):

LoginDto.

Contiene NombreUsuario y Contraseña para autenticación.

RegistroDto.

Combina datos de Persona y Usuario para el registro.

UsuarioResponseDto.

Respuesta del servidor tras login.

Las relaciones entre clases reflejan las llaves foráneas de la base de datos: Persona referencia a Nacionalidad, y Usuario referencia tanto a Persona como a Rol.

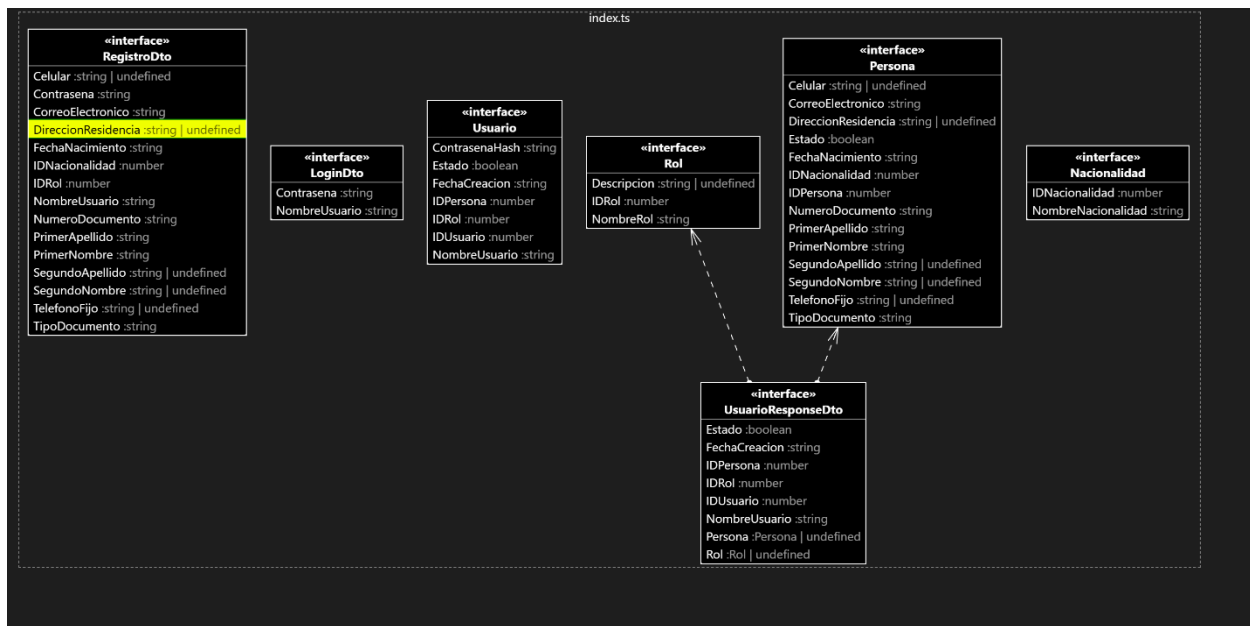


Diagrama de Paquetes

El diagrama de paquetes muestra la organización por módulos del proyecto NotePlus Taller, dividido en dos módulos principales:

Backend:

- app/api — contiene las rutas REST de la API: nacionalidades, personas, roles y auth
- lib — utilidades compartidas: cliente Supabase y middleware CORS
- types — definición de interfaces TypeScript: entidades del dominio y DTOs

Frontend:

- app/(auth) — páginas públicas: login y registro, accesibles sin autenticación
- app/(dashboard) — páginas protegidas: gestión de personas y nacionalidades
- components — componentes reutilizables organizados en tres subcarpetas: ui/ para elementos de interfaz, forms/ para formularios y layout/ para estructura de página

Ambos módulos se despliegan de forma independiente en Vercel y se comunican a través de la API REST.

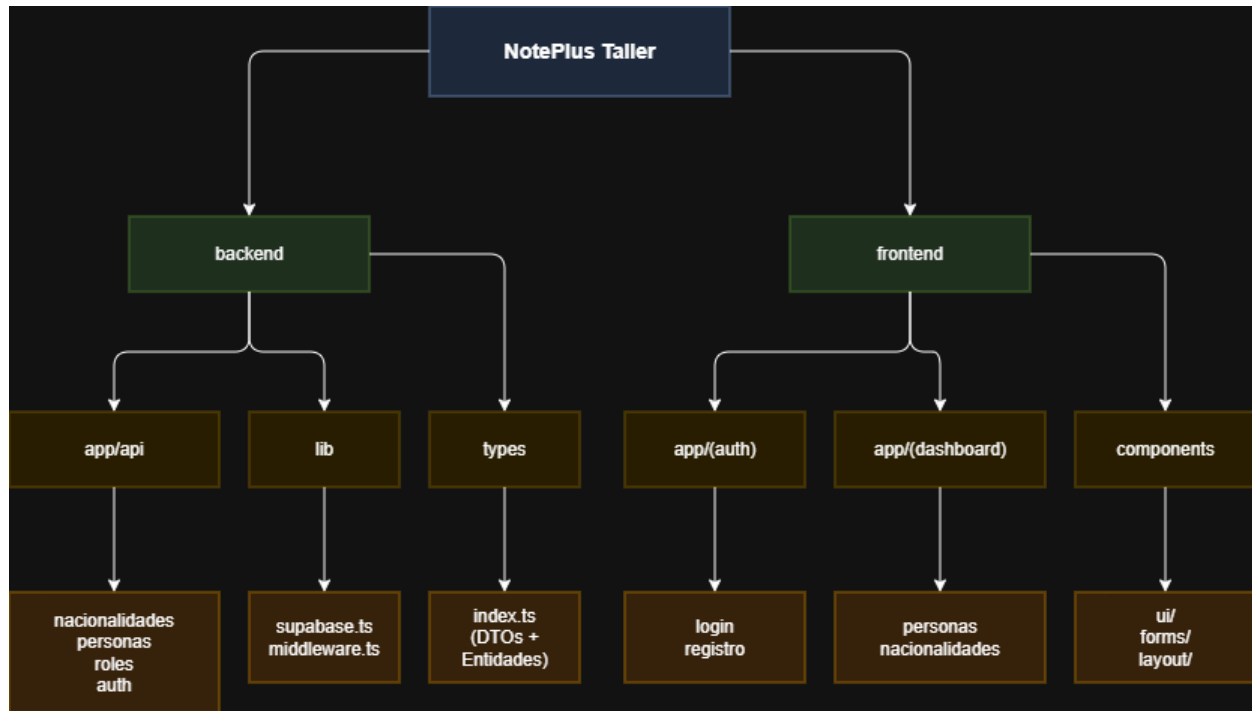


Diagrama de Componentes

El diagrama muestra la arquitectura de componentes del sistema y sus relaciones de dependencia, en este caso organizado en tres capas:

Capa de Base de Datos:

- Supabase — servicio PostgreSQL en la nube que almacena las tablas Nacionalidad, Persona, Rol y Usuario

Capa Backend (Next.js API):

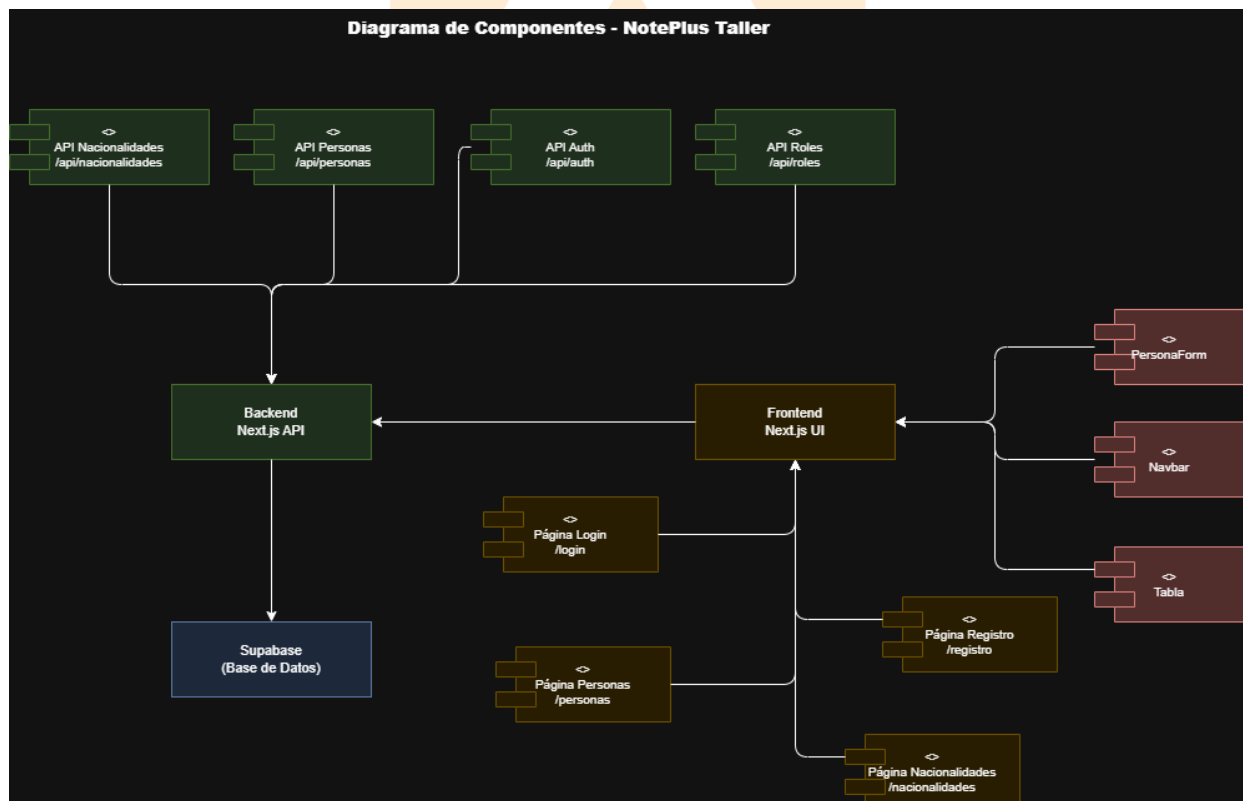
- API Nacionalidades — expone endpoints GET y POST para el catálogo de nacionalidades
- API Personas — expone endpoints CRUD completo para gestión de personas

- API Auth — maneja registro y login con hash de contraseñas bcrypt
- API Roles — expone endpoint GET para el catálogo de roles

Capa Frontend (Next.js UI):

- Página Login y Página Registro — componentes públicos de autenticación
- Página Personas y Página Nacionalidades — componentes protegidos de gestión
- Navbar — componente de navegación con información de sesión
- PersonaForm — formulario reutilizable para crear y editar personas
- Tabla — componente genérico reutilizable para listados

El Frontend se conecta con el Backend a través de HTTP REST, y el Backend se comunica con Supabase mediante el cliente oficial de JavaScript.



Mecanismos de Seguridad

1. Contraseñas protegidas Cuando un usuario se registra, el sistema nunca guarda su contraseña tal como la escribió. En cambio, la convierte en una cadena de caracteres irreconocible (llamada hash) antes de guardarla. El resultado no se puede revertir. Cuando el usuario inicia sesión, el sistema encripta lo que escribió y compara si el resultado es igual al que tiene guardado.

2. Páginas protegidas Las páginas de gestión de personas y nacionalidades solo son accesibles si el usuario ha iniciado sesión. Si alguien intenta entrar directamente escribiendo la URL, el sistema lo redirige automáticamente a la página de login.

3. Claves de acceso separadas El sistema usa dos tipos de llaves para conectarse a la base de datos: una llave maestra que solo usa el servidor (nunca la ve el usuario), y una llave limitada que usa el navegador con permisos reducidos.

4. Credenciales fuera del código Las contraseñas y claves de acceso a los servicios externos no están escritas en el código del proyecto. Se guardan en archivos de configuración separados que nunca se suben a GitHub.

5. Control de origen El backend solo acepta peticiones que vengan del frontend autorizado. Si alguien intenta hacer peticiones desde otro sitio web, el sistema las bloquea automáticamente.

6. Validación de formularios Antes de guardar cualquier dato, el servidor verifica que todos los campos obligatorios estén completos. Si falta alguno, rechaza la operación e informa al usuario qué hace falta.

Capas del Sistema

El sistema está organizado en 3 capas bien definidas:

Capa de Presentación (Frontend) Es lo que el usuario ve y con lo que interactúa directamente en el navegador. Contiene las páginas de login, registro, listado de personas y nacionalidades, y los componentes reutilizables como el formulario de persona, la tabla y la barra de navegación. Desarrollada con Next.js y desplegada en Vercel.

Capa de Negocio (Backend) Es el intermediario entre la interfaz y la base de datos. Recibe las peticiones del frontend, aplica las reglas del negocio (validaciones, hash de contraseñas, control de acceso) y se comunica con la base de datos. Contiene los endpoints de la API REST para personas, nacionalidades, roles y autenticación. Desarrollada con Next.js API Routes y desplegada en Vercel.

Capa de Datos (Base de Datos) Es donde se almacena toda la información del sistema de forma persistente. Contiene las tablas Nacionalidad, Persona, Rol y Usuario con sus relaciones. Gestionada por Supabase (PostgreSQL) en la nube.

Capa	Tecnología	Despliegue
Presentación	Next.js + TailwindCSS	Vercel
Negocio	Next.js API Routes + bcryptjs	Vercel
Datos	PostgreSQL (Supabase)	Supabase Cloud

Metodología de Desarrollo

Para el desarrollo de este taller se aplicó una metodología basada en **desarrollo asistido por IA (Inteligencia Artificial)** mediante prompts:

Herramientas utilizadas:

- **Claude Code** — herramienta de desarrollo asistido por IA instalada en la terminal, usada para generar el código fuente del proyecto mediante prompts en lenguaje natural.
- **Claude.ai** — usado para planificación, resolución de problemas y generación de documentación
- **Supabase** — configurado manualmente para la base de datos en la nube
- **Vercel** — configurado manualmente para el despliegue en producción
- **Postman** — usado manualmente para las pruebas de los endpoints

Fases del Desarrollo

Fase	Descripción	Herramienta
Análisis	Revisión de requerimientos y definición de tablas.	Claude.ai
Configuración	Creación de base de datos y entorno local.	Manual
Backend	Generación de API REST completa.	Claude Code
Frontend	Generación de interfaz de usuario.	Claude Code
Pruebas	Verificación de endpoints.	Postman
Despliegue	Publicación en producción.	Vercel

Documentación	Generación de archivos .md y diagramas	Claude.ai + draw.io
----------------------	--	---------------------

Mapa de Navegación

El sistema tiene dos zonas de navegación claramente separadas:

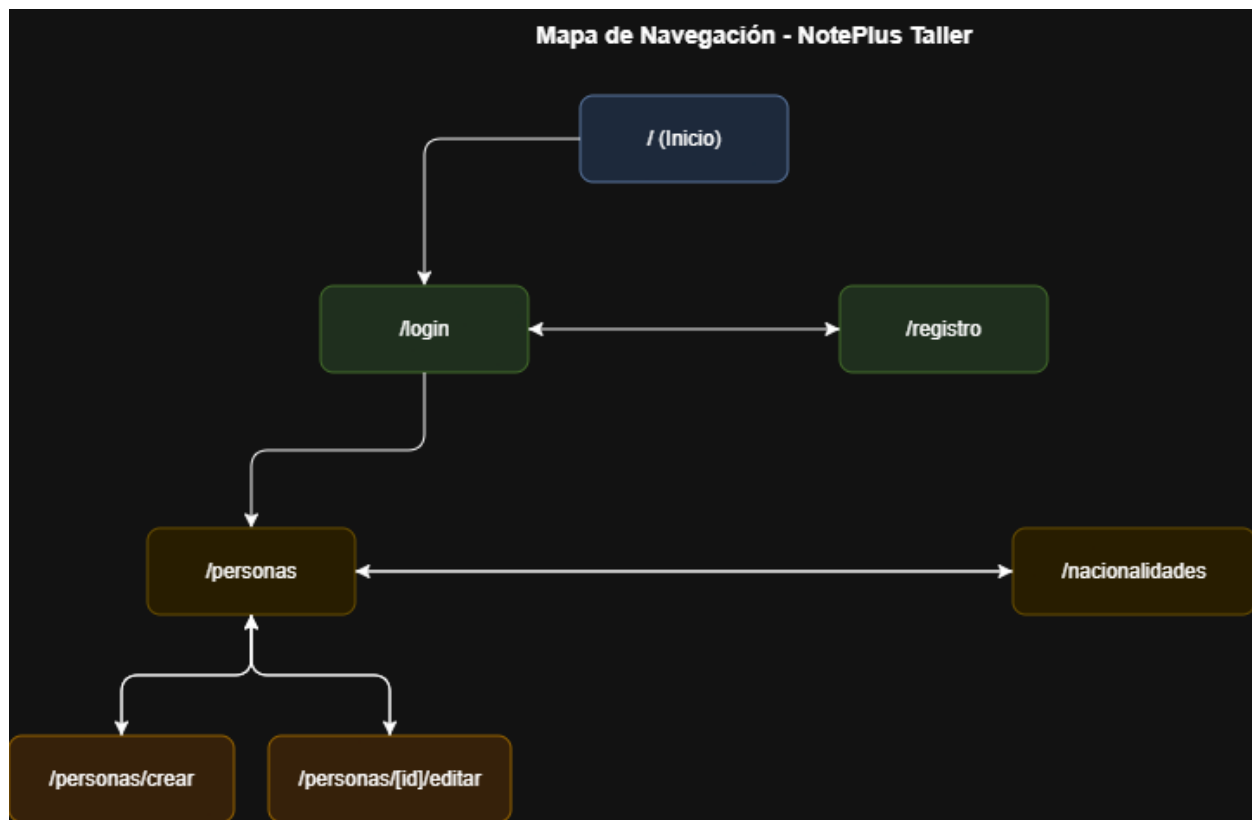
Zona pública (sin autenticación):

- / (Inicio) — punto de entrada, redirige automáticamente a /login
- /login — formulario de inicio de sesión
- /registro — formulario de registro de nueva cuenta. Desde aquí el usuario puede ir a /login y viceversa

Zona privada (requiere autenticación):

- /personas — página principal del dashboard, listado de todas las personas registradas
- /personas/crear — formulario para registrar una nueva persona, regresa a /personas al guardar
- /personas/[id]/editar — formulario para modificar una persona existente, regresa a /personas al guardar
- /nacionalidades — listado de nacionalidades con opción de crear nueva. Se puede navegar entre /personas y /nacionalidades desde la barra de navegación

Si un usuario no autenticado intenta acceder a cualquier página de la zona privada, el sistema lo redirige automáticamente a /login.



Pruebas Unitarias de los Módulos

Para esta evidencia se desarrollaron las pruebas unitarias dentro del entorno de Postman accediendo a cada uno de los endpoints definidos en el sistema y utilizando el método HTTP correspondiente.

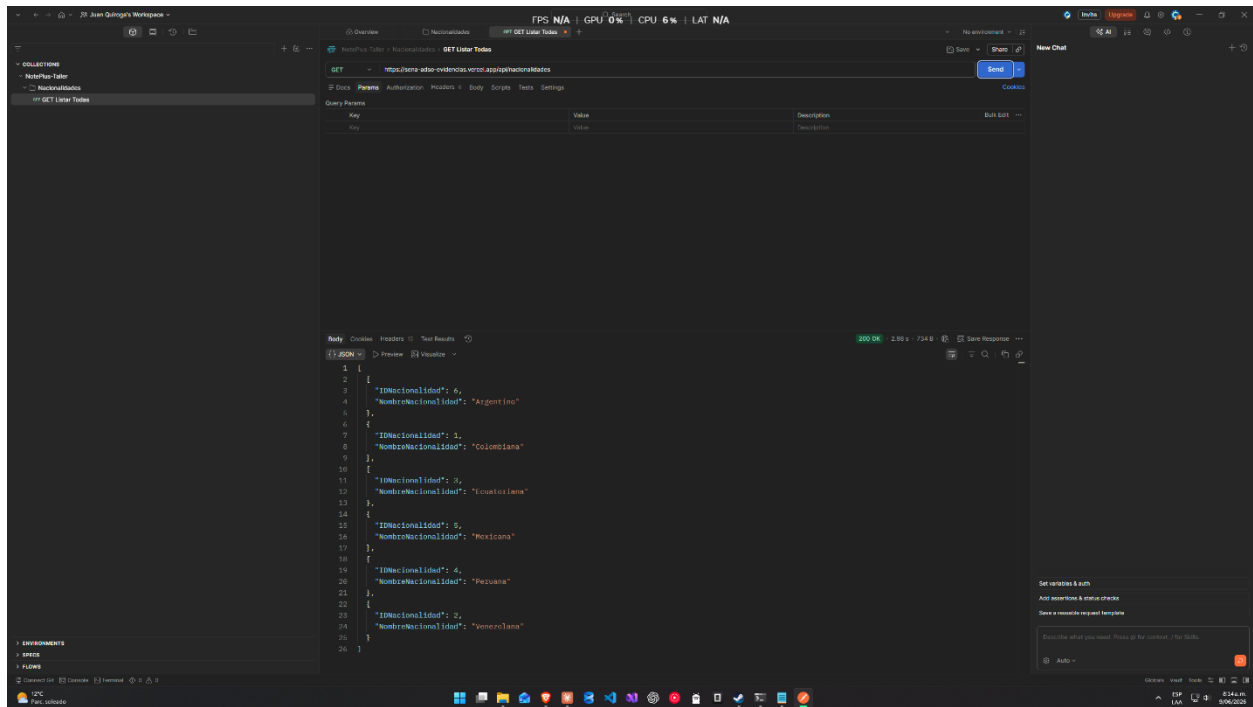
Herramienta: (<https://sena-adso-evidencias.versel.app>)

Módulo 1 – Nacionalidades

Prueba 1 – Listar todas las nacionalidades.

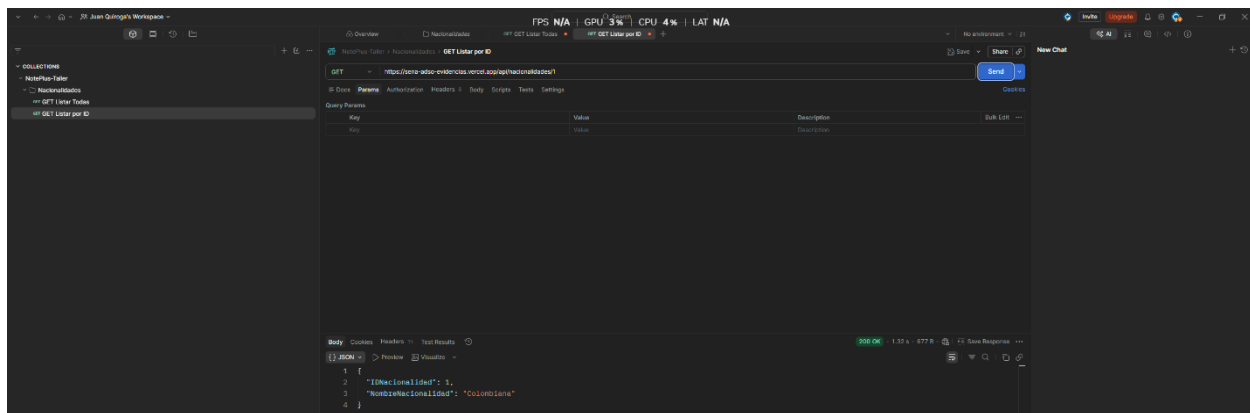
- Método: GET
- Endpoint: /api/nacionalidades
- Resultado esperado: 200 + listado de nacionalidades

- Resultado obtenido: 200 



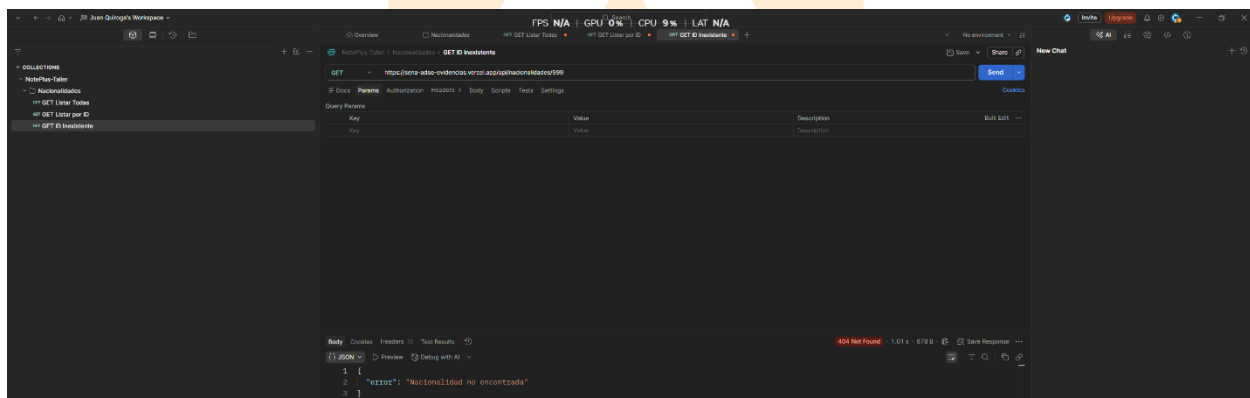
Prueba 2 — Obtener nacionalidad por ID

- Método: GET
- Endpoint: /api/nacionalidades/1
- Resultado esperado: 200 + datos de la nacionalidad
- Resultado obtenido: 200 



Prueba 3 — Obtener nacionalidad con ID inexistente

- Método: GET
- Endpoint: /api/nacionalidades/999
- Resultado esperado: 404 Not Found
- Resultado obtenido: 404

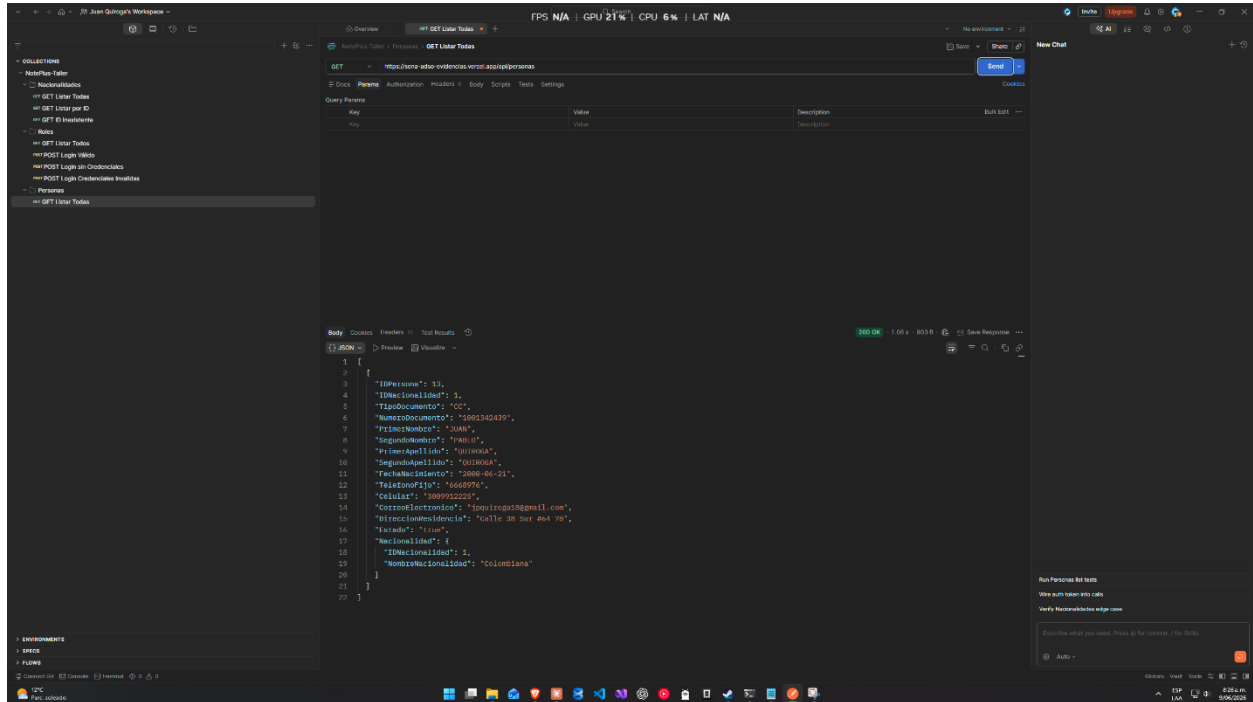


Módulo 2: Personas

Prueba 4 — Listar todas las personas

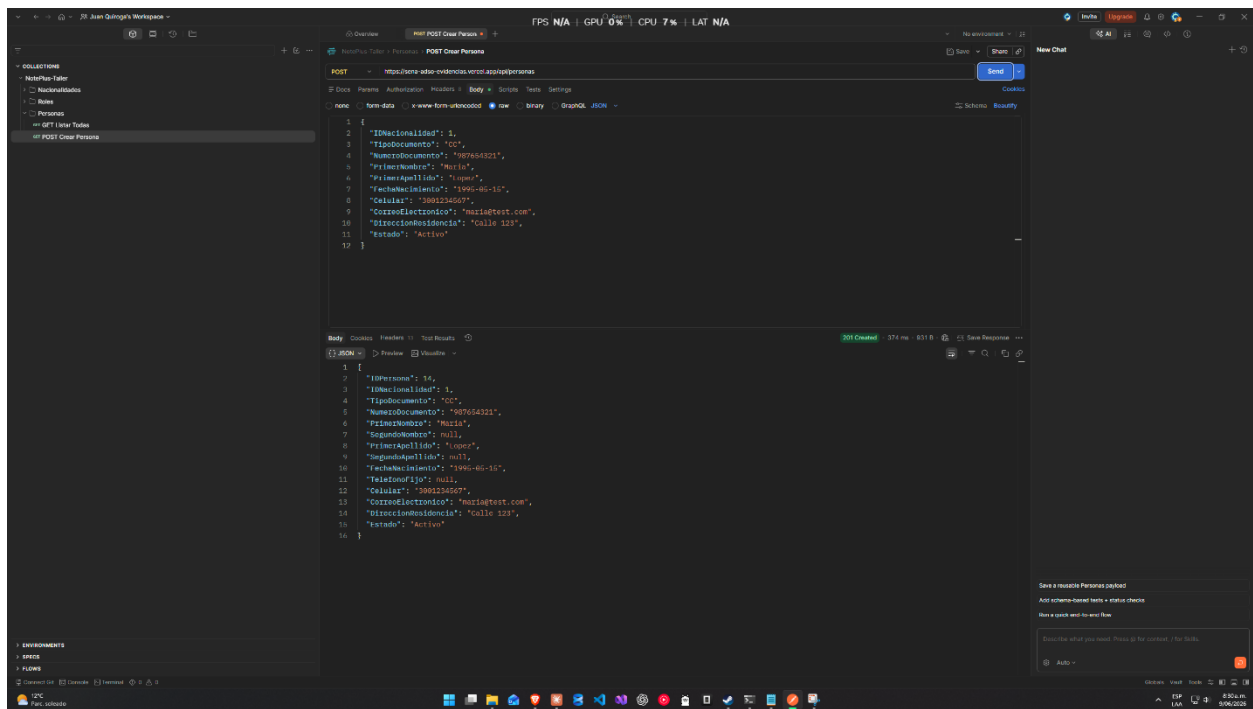
- Método: GET
- Endpoint: /api/personas

- Resultado esperado: 200 + listado de personas
- Resultado obtenido: 200 ✓



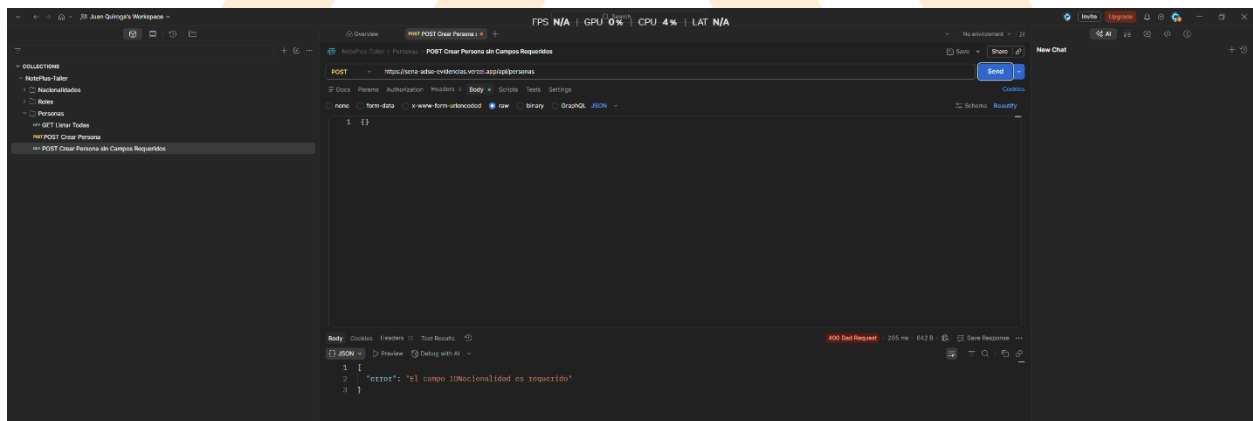
Prueba 5 — Crear persona con datos completos

- Método: POST
- Endpoint: `/api/personas`
- Resultado esperado: 201 + persona creada
- Resultado obtenido: 201 ✓



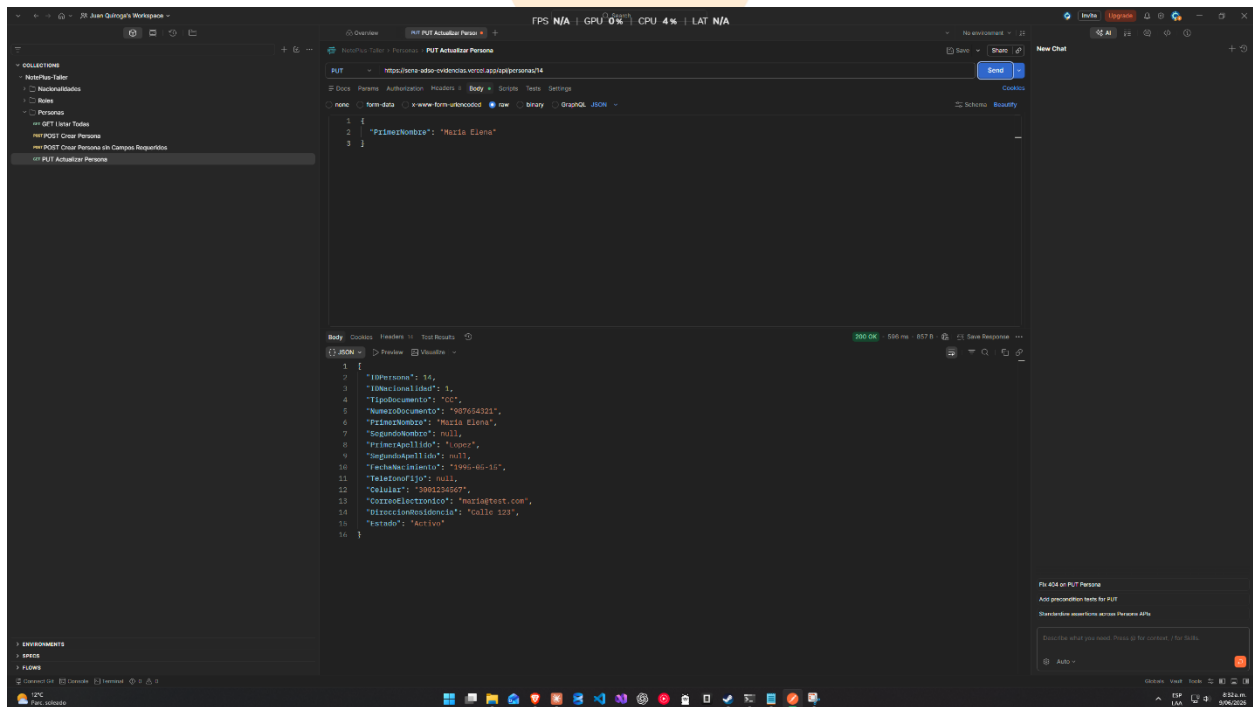
Prueba 6 — Crear persona sin campos requeridos

- Método: POST
- Endpoint: /api/personas
- Resultado esperado: 400 Bad Request
- Resultado obtenido: 400 



Prueba 7 — Actualizar persona existente

- Método: PUT
- Endpoint: /api/personas/[id]
- Resultado esperado: 200 + persona actualizada
- Resultado obtenido: 200 



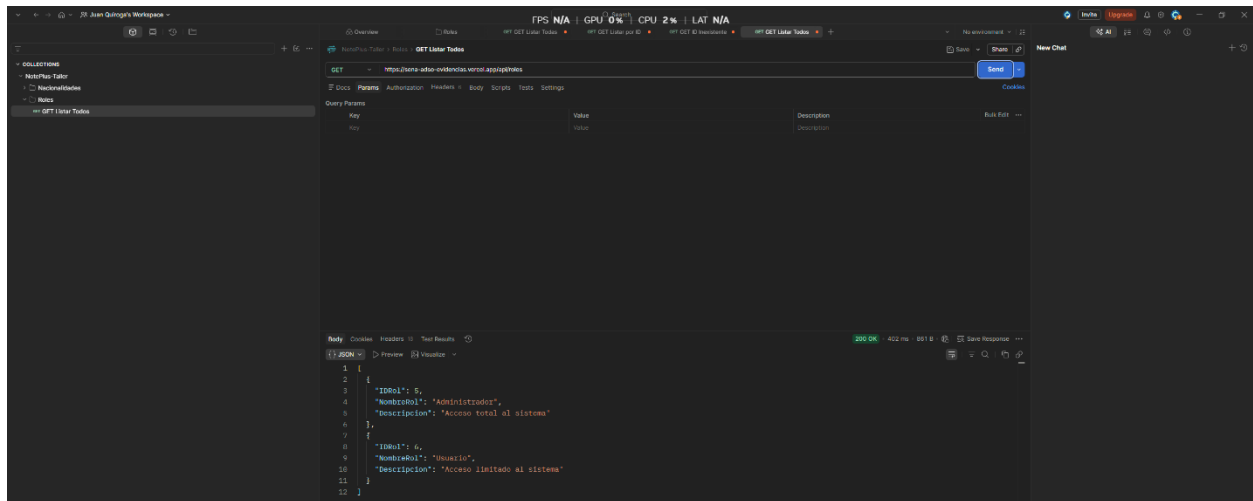
Prueba 8 — Eliminar persona

- Método: DELETE
- Endpoint: /api/personas/[id]
- Resultado esperado: 200 + confirmación
- Resultado obtenido: 200 

Módulo 3: Roles y Autenticación

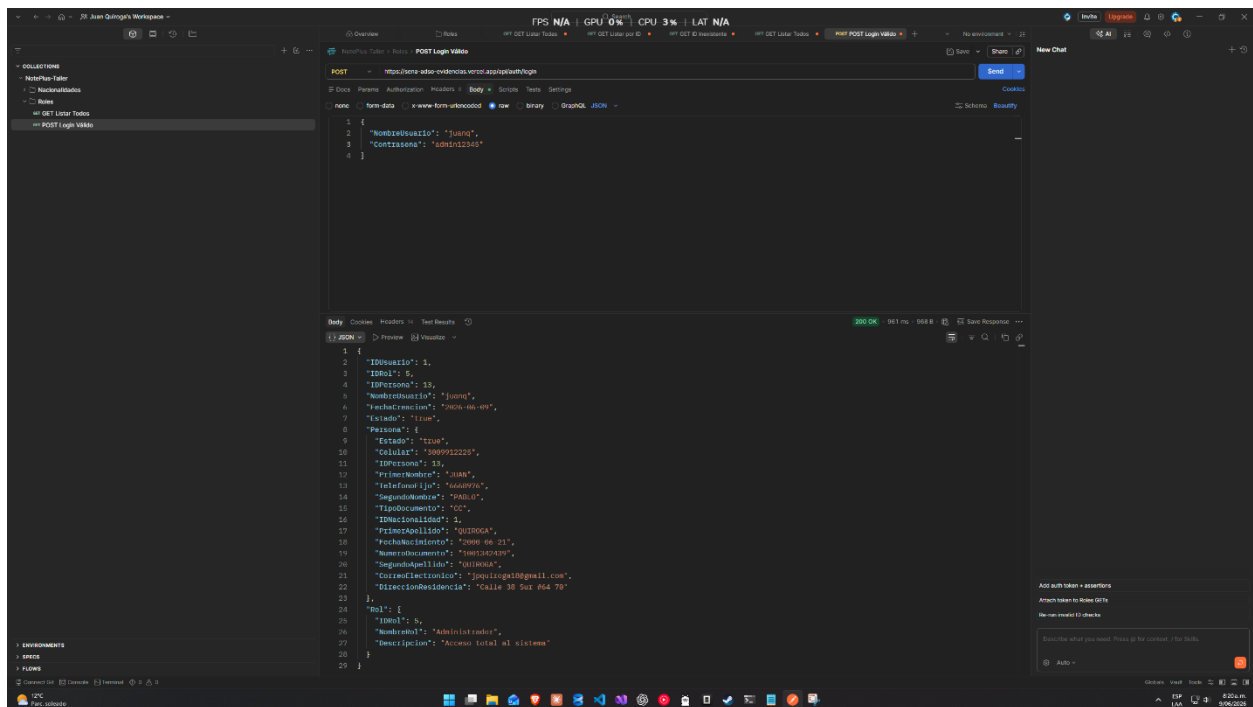
Prueba 9 — Listar todos los roles

- Método: GET
- Endpoint: /api/roles
- Resultado esperado: 200 + listado de roles
- Resultado obtenido: 200 



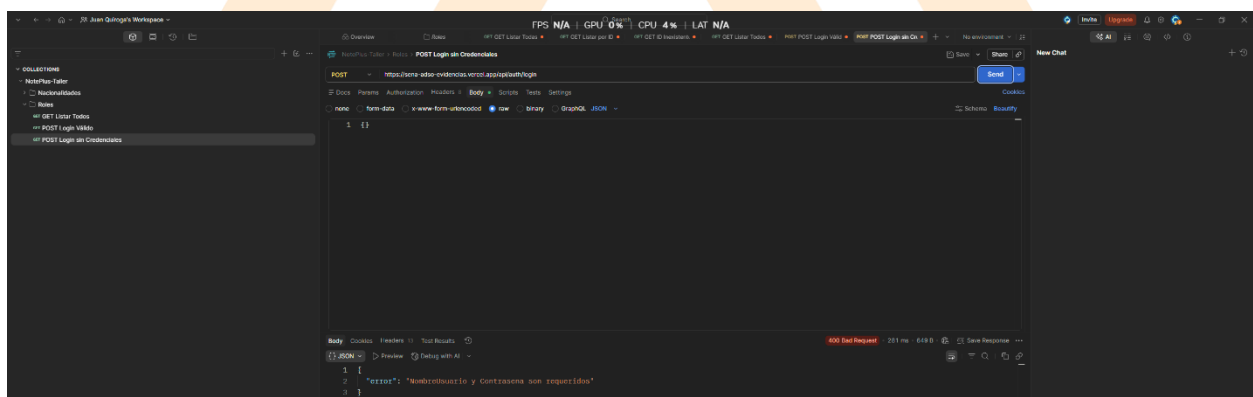
Prueba 10 — Login con credenciales válidas

- Método: POST
- Endpoint: /api/auth/login
- Resultado esperado: 200 + datos del usuario
- Resultado obtenido: 200 



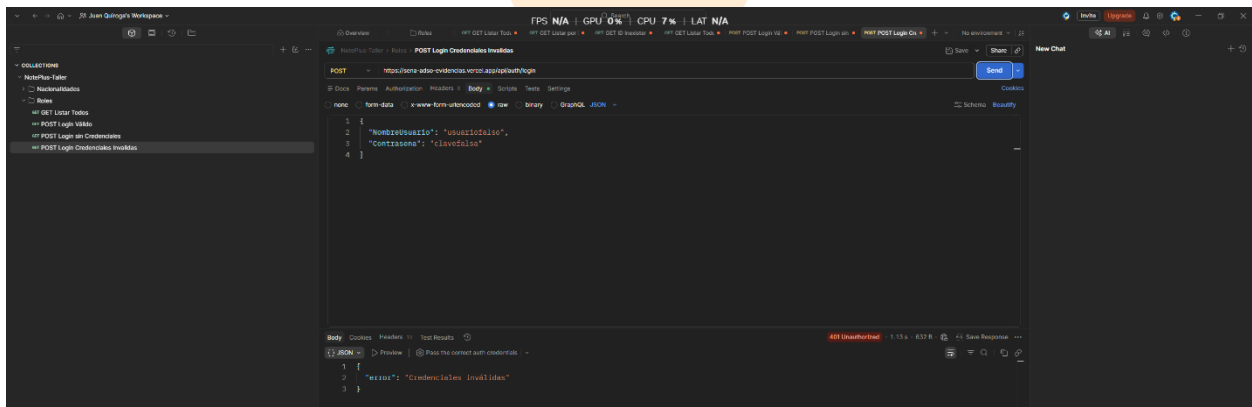
Prueba 11 — Login sin credenciales

- Método: POST
- Endpoint: `/api/auth/login`
- Resultado esperado: 400 Bad Request
- Resultado obtenido: 400 



Prueba 12 — Login con credenciales inválidas

- Método: POST
- Endpoint: /api/auth/login
- Resultado esperado: 401 Unauthorized
- Resultado obtenido: 401 



Colección de Postman

La colección completa de pruebas está disponible en el archivo NotePlus-Taller.postman_collection.json incluido en el repositorio. Para importarla en Postman:

1. Abrir Postman
2. Click en **"Import"**
3. Seleccionar el archivo NotePlus-Taller.postman_collection.json
4. Todas las pruebas están disponibles para ejecutarse nuevamente